

Logan MacAskill

Los Altos, CA | (650) 237-9593 | logan@macaskill.com
logan.macaskill.com | github.com/log2bits | linkedin.com/in/logan-macaskill

ABOUT ME

Third-year CS student with two software engineering internships spent shipping and tuning production systems. I love problem solving, especially when I can take complete advantage of the hardware. I have worked across real-time systems, graphics, AI, and data science, and I am currently building my own Vulkan and C++ deferred renderer.

EDUCATION

University of California, Santa Cruz
B.S. in Computer Science, GPA 3.6/4.0

Santa Cruz, CA
Expected Jun 2028

EXPERIENCE

Software Engineering Intern

Flickr

Jun 2023 – Aug 2023

Mountain View, CA

- Cut compute cost 92% and raised throughput 39% on a Python AWS Lambda service repairing 10 billion photos.
- Tuned memory and thread allocation to the cheapest stable configuration.
- Profiled dozens of Lambda configurations in Splunk, modeling cost per job, throughput, and tail latency.
- Recovered AWS's undocumented vCPU-to-memory scaling ratio by linear regression.

Software Engineering Intern

SmugMug

Jun 2022 – Aug 2022

Mountain View, CA

- Owned a user-facing gallery stats feature end to end in PHP, React and Next.js, from PR review through QA.
- Added daily refresh scheduling, clearer labels and empty states to the stats UI.
- Wrote unit tests and backend changes for a release-critical PHP 8.1 upgrade, landing it on schedule.

PROJECTS

ray-vox: Ray-traced Voxel Renderer | *Rust, WebGPU, Data Structures, Optimization*

2026

- Packed an 84 MB voxel scene into 16 MB with a custom sparse-voxel structure, beating zstd at max (19 MB).
- Rays walk the compressed form directly, with no decompression step.
- Removed every child pointer with per-node bitmasks and popcount, packing each node offset into one 32-bit word.
- Traversal stays branch-light, so GPU warps stay coherent.
- Designed it as a GPU acceleration structure with level-of-detail and matching on-disk and in-GPU layouts.

Sanity Check: Logic Puzzle Game | *Unity, C#, Constraint Solving, Procedural Generation*

2026

- Built the puzzle engine for a five-person, one-week game jam.
- Encoded every possible answer as one bit in a hand-written bitset, so solving a room is set intersection.
- Generated puzzles that are provably unique and minimal, rejecting any room where a clue could be dropped.
- Hand-wrote the set layer with 64-bit words and SWAR popcount, scoring 20,000 candidate rooms per puzzle.
- Tuned five difficulty tiers on how many statements must combine before the first deduction.
- Also owned the game's lighting, post-processing and camera feel.

Real-time Dielectric Spectral Raymarcher | *GLSL, Spectral Rendering, Sampling, GPU*

2026

- Rendered a physically based dispersive diamond in a single real-time GLSL shader, live in a browser tab.
- Resolved 65,536 wavelengths from two samples per pixel by scheduling them across a 16x16 Bayer dither.
- Replaced raymarching with exact ray-plane intersection across the icosahedron's 20 faces.
- Peeled exact Fresnel energy per facet instead of sampling one path, staying noise-free at one sample.
- Terminated each ray below 4% trapped energy.
- Kept every pixel a self-contained shader invocation, with no frame history.

Crowd Surfers: Real-time 3D Game | *Godot, Shaders, Lighting, Architecture*

2025 – 2026

- Convinced a 100-person student team to rebuild the renderer mid-project by building a prototype instead of arguing.
- Moved it from faked-depth sprites to true 3D with an angled orthographic camera.
- Built an occlusion shader fading buildings as the player skates behind them, with a frustum test and dithered alpha.

Coaxial Swerve Drive | *Java, Control Theory, Computer Vision*

2023 – 2024

- Led an off-season FRC team to a working coaxial swerve prototype in one month, four months ahead of schedule.
- Built the CNC chassis, the electronics, and 2,000 lines of Java.
- Held sub-centimeter localization through wheel slip by fusing encoders, gyro and AprilTag vision in a Kalman filter.
- Ran per-module PID and feedforward on a 1 kHz control loop after self-teaching graduate-level control theory.
- Simulated torque curves, inertia and brownouts on the production code, deriving current limits from tire friction.

TECHNICAL SKILLS

Languages: Rust, C++, C#, Python, Java, TypeScript / JavaScript, PHP, GLSL / WGSL

Graphics & GPU: Vulkan, WebGPU, Ray Tracing, Real-Time Rendering, Rasterization, Shaders, Level-of-Detail

Tools & Cloud: Git, Linux, AWS (EC2, Lambda, Graviton), Docker, OpenCV, OpenAI API, Godot, Unity, React / Next.js

Concepts: Performance Optimization, GPU Compute, Computer Vision, Pose Estimation, LLMs / Prompt Engineering, Control Theory